Content From Slides:
The essential components of a Microkernel are:

- IPC
- Memory Manager
- CPU Scheduler

Everything else is userland drivers, file system ops, and applications

The process manager handles context switching and scheduling metrics (utilization etc). It does *not* handle allocation of memory to a process. This is the memory management unit

Context Switch Time depends on:

- PCB complexity within OS (process control block)
- How many contexts are being loaded
- The complexity of the register file

Message Passing is better for smaller transfers. It is generally slower though because the data must be copied. It cannot directly be accessed by 2 processes. Message passing is better in distributed systems where there are discrete systems that may not have direct memory access, message passing conforms better with packet protocols like TCP or ARP.

Shared Memory should exist within the process that creates it.

- ftok(filename, projectid) ← uses a file as an exchange of SHM metadata. Returns a key
- shmget(key, size, flags) ← uses the key to create a SHM region, if there already exists a region associated with the key, it returns the same region as what is used by the second region. Returns an ID used to attach and detach
- shmat(id, pointer, flags) ← takes the id from smget and points the given pointer to the region
- shmdt(pointer) ← detaches

Message Passing:

- Ftok
- msgget(key, flags)
- msgsnd(id, &msg, size, 0)
- msgrecv(id, &msg, size, 1, 0)
- msgctl(id, IPC_RMID, NULL) ← destroy message queue

Message passing is more scalable on distributed memory systems where each CPU has its own memory, since this enhances memory bandwidth which unclutters the interconnect between the CPUs. Message passing is also more common in Operating Systems.

In *indirect message links* the link binds together 2 or more processes. They can be bidirectional or unidirectional.

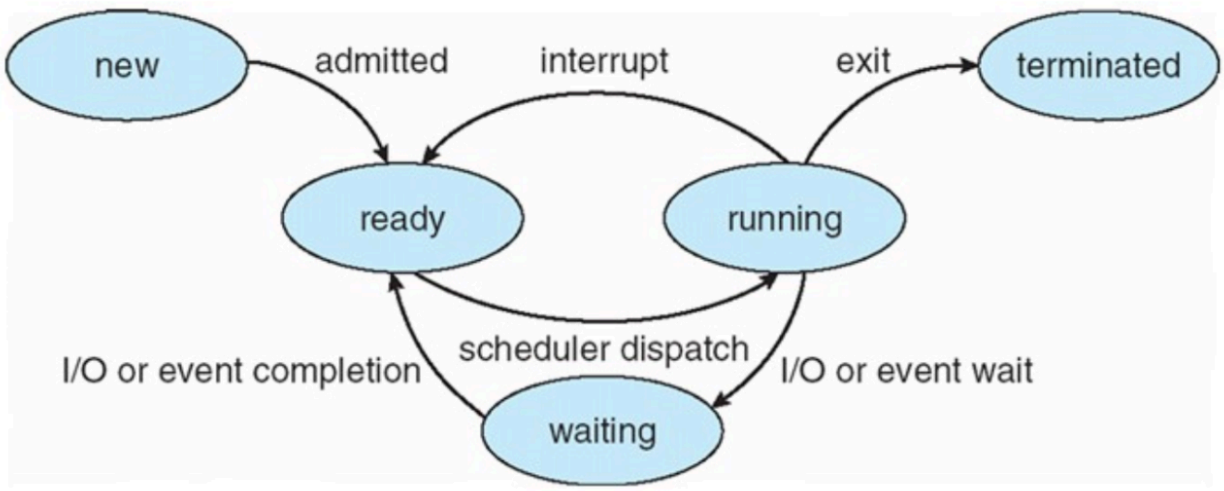
A mailbox can be shared by any number of processes. Only one process at a time can do a receive operation
I think a link under *direct message passing* is bidirectional (might need to check with lecture). Also under *direct* there should be exactly one link between processes.

SHM POSIX API:

- shmopen ← create shm
- close() ← close the object
- mmap() ← mapped the shared region into VA space
- munmap() ← unmap the shared region
- ftruncate() ← adjust the size of the shared memory region
- shm_unlink() ← remove a shared object name (prevents future processes from opening it; however it is not destroyed until all processes no longer use it)

RabbitMQ:

- Publisher attaches to exchange
- Exchange attached to certain queues
- Subscribers read from queues
    - When sending a message, determine an exchange, a queue, and a payload



Multi-Level Queues:

- Partition the queues into 2 parts
    - 1 for interactive processes
    - 1 for batch processes
- User different scheduling policies with each queue
- Each queue gets timesliced onto the processor to some proportion

Multi-Level Feedback Queues:

- Processes can move between queues

- There's a diagram of which queues in the multi-level queues can starve, this is likely important. Hopefully there's detail in the surveys or panopto (Lecture 11)

Multi-Processor Scheduling: (which processor should execute the process)

- Centralized Master-Slave architecture (master assigns process to slave processor)
- Distributed Methodology:
    - Pooled Resources
- Methods of Scheduling:
    - Load sharing
        - Process not assigned to one particular processor
        - Good for distributed methodology
        - Not good at allowing concurrency since threads may not execute at once
    - Gang Scheduling
        - Related threads run on a certain processors at once
    - Dedicated Processor Assignment
        - Process assigned to one particular processor
    - Dynamic Scheduling
        - Process can be moved around during execution

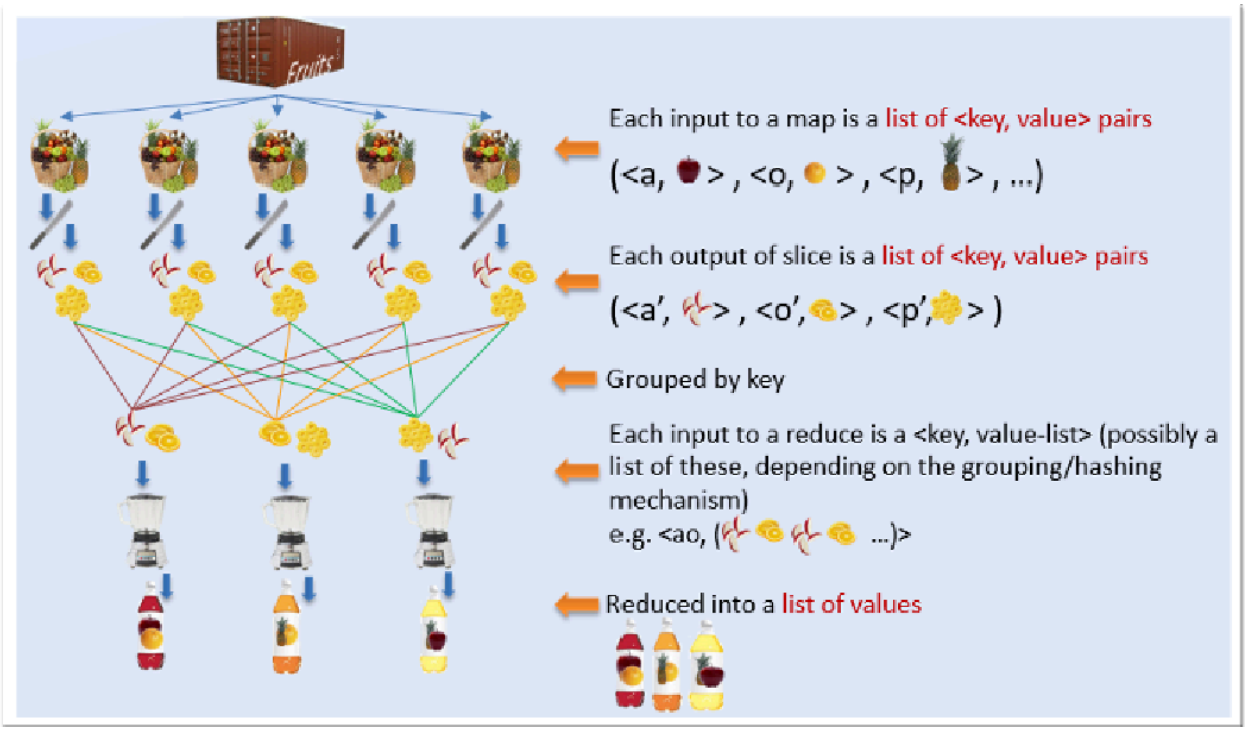
Real Time Scheduling Methods:

- Static Table Driven
    - Plan made in advance for scheduling periodic tasks
    - Changes means schedule must be entirely rewritten
- Static Priority-Driven Preemptive
    - Give real-time processes high priority so they can preempt
- Dynamic Planning Based
    - Try to revise schedule when new task arrives
- Dynamic Best Effort
    - Assign priorities
        - Ex: Earliest Deadline First
    - Very popular with real time systems

Rate Monotonic Scheduling:

- Schedules periodic tasks with a static priority policy with preemption

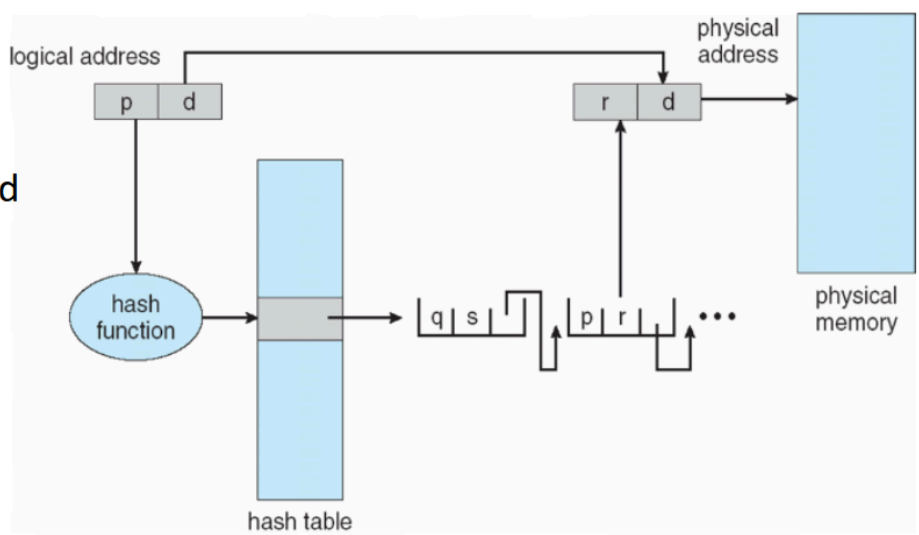
Map Reduce:



I feel like Dr. Qin skipped. YARN, so I will too. YOLO

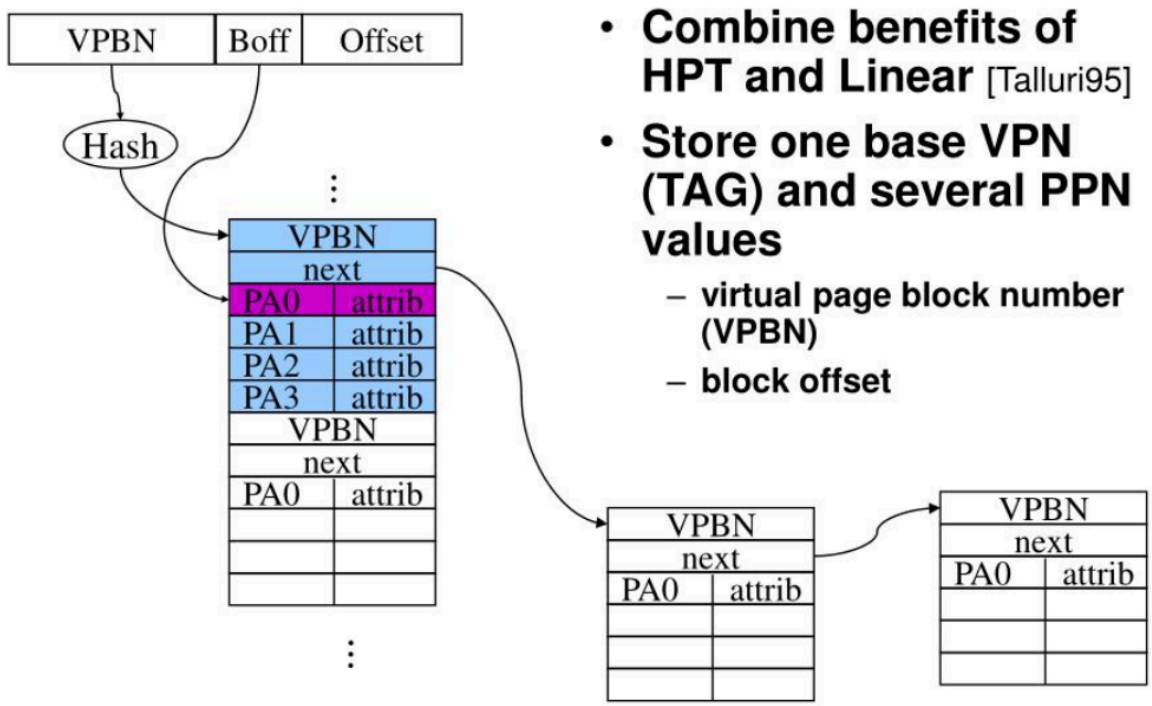
PTBR (page table base register) points to the page table in memory. PTLR (page table length register) records the size of the page table.

The virtual page number is hashed into a page table



Clustered Page Tables:

- Each entry represents a group of pages instead of a single one



Inverted Page Table Architecture:

- Each entry is pid + page# + offset
- Each frame in memory is represented once in page table
- Lower memory usage
- Higher search time

Content From Class Surveys:
See quizlet
Content From Quizzes:
See quizzes
Content From Projects:
Fill in programming topics within instructor study guide
Content From Homework:
See HW (allowed in exam)

Unix Pipes
int fd[2];
pipe(fd)
Read end = fd[0]
Write end = fd[1]

write(fd[1], char* message, strlen(message) + 1 (for null byte))
read(fd[0], char* message, BUFFER_SIZE)

Each side of pipe must be closed individually:

close(fd[0]);

int pid = fork();
pid == 0 is the child process; pid > 0 is the parent

Condition Variables:
struct lock* mutex;
Mutex = lock_create("mutex name");

lock_acquire(mutex)
lock_release(mutex)

How to use condition variables to wait?

```
void producer(char c) {
    lock_acquire(mutex);
    while (count == SIZE) {
        cv_wait(notFull, mutex);
    }
    count++;
    buffer[head] = c;
    head++;
    if (head == SIZE) {
        head = 0;
    }
    cv_signal(notEmpty, mutex);
    lock_release(mutex);
}
```

AUBURN UNIVERSITY
SAMUEL GINN COLLEGE OF ENGINEERING

Cv_wait:
- Atomically releases the associated mutex
- Puts the calling thread to sleep
- When awakened, it reacquires the mutex before returning

Cv_signal:
- Awakens waiting threads

PThread
pthread_create(pthread_t* handle, const pthread_attr_t* attribute, pthread function (function pointer), function arg list ..)

pthread_exit() ← used when the worker thread finished to forward returns
pthread_join() ← stalls the master thread until the worker thread finishes/exits